DECK 1 OF 3

LINUX SERVER SERVICES

Common Linux network services: what they do, how to run them, and how to check them

SSH 22 | HTTP 80 | DNS 53 | SMB 445 | SQL 3306

Presented by: Tyrone Justine A. Medina

OVERVIEW

Agenda

- **Part 1: Concepts**
  Services, ports, protocols, managing services with systemd
- **Part 2: Web server**
  Nginx and Apache: virtual hosts, testing, HTTPS
- **Part 3: FTP server**
  vsftpd: active vs passive, users, chroot, FTPS, testing
- **Part 4: Database server**
  MariaDB and PostgreSQL: users, privileges, backups, remote access
- **Part 5: Proxy server (Squid)**
  Forward proxy, ACLs, authentication, caching, logs
- **Part 6: Other services and audit**
  SSH, DNS, DHCP, Samba/NFS, mail, finding what is running

What Is a Network Service?



- **Service (daemon)**
  a background program that listens for requests and answers them
- **Port**
  the numbered door a service listens on (0-65535)
- **Protocol**
  the rules of the conversation (TCP, UDP, HTTP, DNS)
- Every listening service is part of the attack surface

Ports: Three Ranges

Range	Name	Used for	Notes
0 - 1023	Well-known	SSH 22, HTTP 80, HTTPS 443, DNS 53	Binding needs root or a capability
1024 - 49151	Registered	MySQL 3306, PostgreSQL 5432, app ports	Assigned by IANA, usable by normal users
49152 - 65535	Dynamic / ephemeral	Client side of a connection	Linux default is 32768-60999

> *See your own range: cat /proc/sys/net/ipv4/ip_local_port_range*

Service Map: Ports and Purpose

Service	Common software	Default port	Purpose
SSH	OpenSSH	`TCP 22`	Secure remote administration
Web	Nginx, Apache	`TCP 80 / 443`	Websites and APIs
DNS	BIND9, dnsmasq	`UDP/TCP 53`	Names to IP addresses
DHCP	isc-dhcp, dnsmasq	`UDP 67 / 68`	Automatic IP assignment
File sharing	vsftpd, Samba, NFS	`21, 445, 2049`	Share and transfer files
Mail	Postfix, Dovecot	`25, 587, 993`	Send and receive email
Database	MariaDB, PostgreSQL	`3306 / 5432`	Store application data

Managing Services with systemd

SYSTEMD

```
# What is running?
systemctl list-units --type=service --state=running

# Control a service
sudo systemctl start nginx
sudo systemctl stop nginx
sudo systemctl restart nginx     # full restart
sudo systemctl reload nginx      # re-read config

# Start at boot
sudo systemctl enable nginx
sudo systemctl disable --now cups
systemctl is-enabled nginx

# Status and logs
systemctl status nginx
journalctl -u nginx -f
```

- **enable vs start**
  start runs it now. enable makes it start at boot. They are independent.
- **reload vs restart**
  reload applies config without dropping connections, when supported.
- **journalctl -u**
  Per-service logs. First stop when something fails.

Anatomy of a Unit File

CUSTOM SERVICE

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target

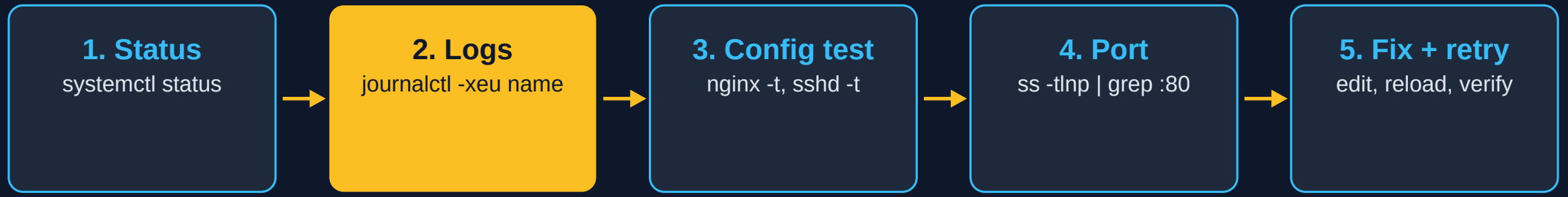
[Service]
User=svc-app
ExecStart=/opt/myapp/bin/myapp
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target

# Then:
sudo systemctl daemon-reload
sudo systemctl enable --now myapp
```

- **[Unit]**
  Description and ordering (After=, Requires=).
- **[Service]**
  Who runs it, what command, how to restart.
- **[Install]**
  Which target pulls it in at boot.
- Own files go in /etc/systemd/system/, packages use /lib/systemd/system/

Troubleshooting a Failed Service



- **Address already in use**
  another process owns the port, find it with ss -tlnp
- **Permission denied**
  file ownership, a port below 1024, or an AppArmor/SELinux denial
- **Exit code in status**
  the real reason is almost always in journalctl, not in status
- Always test the config before you reload a production service

Web Servers: Nginx vs Apache

Nginx

- Event-driven, very low memory per connection
- Excellent for static files and reverse proxy
- Config in nginx.conf and sites-available/
- No .htaccess, changes need a reload

Apache (apache2)

- Process/thread model with many loadable modules
- Per-directory control with .htaccess
- a2ensite, a2enmod, apache2ctl configtest
- Great for classic PHP and shared hosting

Ports 80 (HTTP) and 443 (HTTPS). Use TLS from Let's Encrypt / certbot and hide the version with server_tokens off;

Nginx: A Basic Server Block

NGINX

```
# /etc/nginx/sites-available/example
server {
    listen 80;
    server_name example.com www.example.com;
    root /var/www/example;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}

# Enable and apply
sudo ln -s /etc/nginx/sites-available/example \
           /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

- **server_name**
  Which hostname this block answers for.
- **root / index**
  Folder to serve and default file.
- **location**
  Rules for URL paths, try_files falls back to 404.
- Never reload before nginx -t passes

Apache: Virtual Host and LAMP

APACHE2

```
sudo apt install apache2

# /etc/apache2/sites-available/example.conf
<VirtualHost *:80>
    ServerName example.com
    DocumentRoot /var/www/example
    ErrorLog ${APACHE_LOG_DIR}/example-error.log
    CustomLog ${APACHE_LOG_DIR}/example-access.log combined
</VirtualHost>

sudo a2ensite example
sudo a2dissite 000-default
sudo apache2ctl configtest
sudo systemctl reload apache2
```

- **VirtualHost**
  one Apache can host many sites, chosen by ServerName
- **a2ensite / a2enmod**
  enable sites and modules with symlinks
- **LAMP stack**
  Linux + Apache + MariaDB + PHP: apt install php libapache2-mod-php php-mysql
- Delete any info.php test page after checking PHP

Web: Test, Logs and HTTPS

TEST AND LOGS

```
# Test it
curl -I http://localhost
curl -I https://example.com
curl -s -o /dev/null -w "%{http_code}\n" http://localhost

# Logs
sudo tail -f /var/log/nginx/access.log
sudo tail -f /var/log/nginx/error.log

# Top requested URLs
awk '{print $7}' /var/log/nginx/access.log \
  | sort | uniq -c | sort -nr | head
```

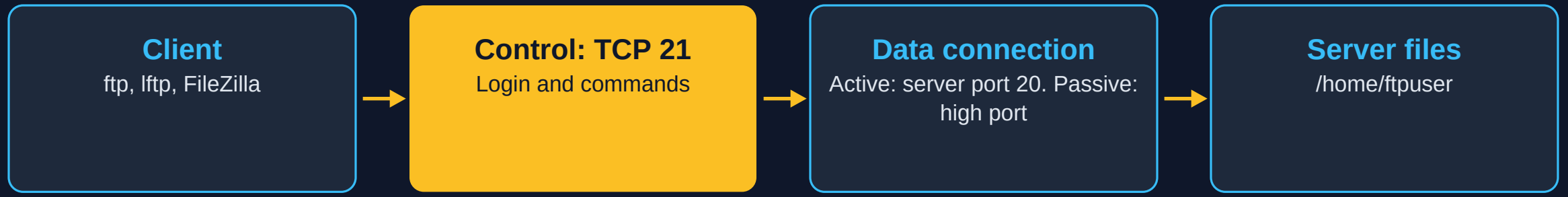
FREE TLS (CERTBOT)

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d example.com -d www.example.com

# Renewal is automatic. Test it:
sudo certbot renew --dry-run
sudo systemctl list-timers | grep certbot

# Check the certificate
openssl s_client -connect example.com:443 \
  -servername example.com </dev/null | head
```

FTP Server: How FTP Works



- **Active mode**
  the server connects back to the client, often blocked by client firewalls
- **Passive mode**
  the client connects to a port the server opens, the usual choice today
- **vsftpd**
  Very Secure FTP Daemon, the common FTP server on Ubuntu and Debian
- Plain FTP sends passwords in clear text, use FTPS or SFTP outside a lab

vsftpd: Install and Configure

VSFTPD

```
sudo apt install vsftpd
sudo cp /etc/vsftpd.conf /etc/vsftpd.conf.bak

# /etc/vsftpd.conf
anonymous_enable=NO
local_enable=YES
write_enable=YES
local_umask=022
chroot_local_user=YES
userlist_enable=YES
userlist_file=/etc/vsftpd.userlist
userlist_deny=NO
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40100
xferlog_enable=YES

sudo adduser ftpuser
echo ftpuser | sudo tee -a /etc/vsftpd.userlist
sudo systemctl restart vsftpd
```

- **anonymous_enable=NO**
  no anonymous logins
- **chroot_local_user**
  locks each user inside their home folder
- **userlist**
  only the names in the file may log in (whitelist)
- **Passive range**
  must match the firewall rules
- With chroot the home root must not be writable: create an uploads/ folder inside it

FTP: Secure It, Open the Firewall, Test It

FTPS (TLS)

```
sudo openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout /etc/ssl/private/vsftpd.pem \
  -out /etc/ssl/private/vsftpd.pem

# /etc/vsftpd.conf
ssl_enable=YES
rsa_cert_file=/etc/ssl/private/vsftpd.pem
rsa_private_key_file=/etc/ssl/private/vsftpd.pem
force_local_logins_ssl=YES
force_local_data_ssl=YES
```

FIREWALL AND TEST

```
sudo ufw allow 21/tcp
sudo ufw allow 40000:40100/tcp
ss -tlnp | grep :21

# self-signed cert? in lftp first run:
# set ssl:verify-certificate no
lftp -u ftpuser localhost
ls
put report.txt
get report.txt
bye

sudo tail -f /var/log/vsftpd.log
```

> *Best practice: prefer SFTP (OpenSSH, port 22). Use FTPS only when a client or partner requires FTP.*

Database Server: MariaDB Setup

MARIADB

```
sudo apt install mariadb-server
sudo systemctl status mariadb
sudo mariadb-secure-installation

# Log in as root through the local socket
sudo mariadb

SHOW DATABASES;
SELECT VERSION();

# Listening where?
ss -tlnp | grep 3306

# /etc/mysql/mariadb.conf.d/50-server.cnf
bind-address = 127.0.0.1
```

- **Purpose**
  stores application data in tables, queried with SQL
- **Port**
  TCP 3306 (MariaDB / MySQL)
- **Secure install**
  sets the root auth, removes test database and anonymous users
- **bind-address**
  127.0.0.1 keeps it local. Change it only when remote access is truly needed.

MariaDB: Users, Privileges and Backups

SQL

```
CREATE DATABASE appdb CHARACTER SET utf8mb4;
CREATE USER 'app'@'localhost'
  IDENTIFIED BY 'ChangeMe!';
GRANT SELECT,INSERT,UPDATE,DELETE
  ON appdb.* TO 'app'@'localhost';
SHOW GRANTS FOR 'app'@'localhost';

USE appdb;
CREATE TABLE users (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50));
INSERT INTO users (name) VALUES ('alice');
SELECT * FROM users;
```

BACKUP, RESTORE, REMOTE

```
# Backup and restore
mysqldump -u root -p appdb > appdb.sql
mysql -u root -p appdb < appdb.sql

# Remote app server (10.0.0.20) needs:
# 1) bind-address = 0.0.0.0
# 2) a user for that host
CREATE USER 'app'@'10.0.0.20'
  IDENTIFIED BY 'ChangeMe!';
# 3) a firewall rule from that host only
sudo ufw allow from 10.0.0.20 to any port 3306
```

> *Give each application its own user with only the privileges it needs. Never let an app connect as root.*

PostgreSQL: Roles, Remote Access, Backups

PSQL

```
sudo apt install postgresql
sudo systemctl status postgresql

sudo -u postgres psql
CREATE ROLE app LOGIN PASSWORD 'ChangeMe!';
CREATE DATABASE appdb OWNER app;
\l          -- list databases
\du         -- list roles
\c appdb    -- connect
\q
```

CONFIG AND BACKUP

```
# postgresql.conf
listen_addresses = 'localhost'

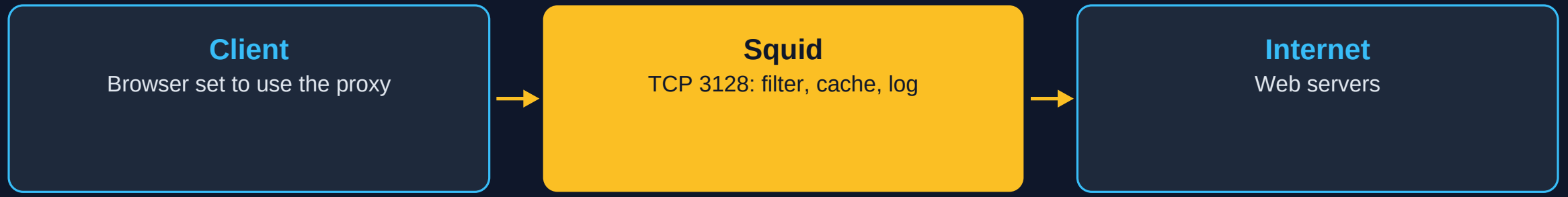
# pg_hba.conf: who may connect, from where
host appdb app 10.0.0.0/24 scram-sha-256

sudo systemctl restart postgresql
ss -tlnp | grep 5432

# Backup and restore
sudo -u postgres pg_dump appdb > appdb.sql
sudo -u postgres psql appdb < appdb.sql
```

Port 5432. pg_hba.conf is checked top to bottom, the first matching line decides.

Proxy Server: What Squid Does



- **Forward proxy**
  sits in front of clients: access control, caching, logging (Squid)
- **Reverse proxy**
  sits in front of servers: TLS, load balancing (Nginx, HAProxy)
- **Why use Squid**
  save bandwidth with caching, block sites, require login, record who browsed what
- Default port 3128, config in /etc/squid/squid.conf

Squid: Install and Basic Configuration

SQUID.CONF

```
sudo apt install squid
sudo cp /etc/squid/squid.conf /etc/squid/squid.conf.bak

# /etc/squid/squid.conf (key lines)
http_port 3128
acl localnet src 192.168.1.0/24
acl SSL_ports port 443
acl Safe_ports port 80 21 443
acl CONNECT method CONNECT

http_access deny !Safe_ports
http_access deny CONNECT !SSL_ports
http_access allow localnet
http_access deny all

cache_mem 256 MB
cache_dir ufs /var/spool/squid 2000 16 256

sudo squid -k parse
sudo systemctl restart squid
sudo ufw allow from 192.168.1.0/24 to any port 3128 proto tcp
```

- **acl**
  names a group: a network, a domain, a port, a time
- **http_access**
  allow or deny rules, read top to bottom, first match wins
- **deny all**
  always last, nothing is allowed by accident
- **cache_dir**
  disk cache: 2000 MB here
- squid -k parse checks the config before you restart

Squid: Block Sites, Time Rules, Login

ACLS (ADD BEFORE DENY ALL)

```
# /etc/squid/blocked_sites.txt
.facebook.com
.tiktok.com

acl blocked dstdomain "/etc/squid/blocked_sites.txt"
http_access deny blocked

# Block only during work hours
acl work_hours time MTWHF 08:00-17:00
acl social dstdomain .youtube.com
http_access deny social work_hours
```

REQUIRE A USERNAME

```
sudo apt install apache2-utils
sudo htpasswd -c /etc/squid/passwords alice

auth_param basic program \
  /usr/lib/squid/basic_ncsa_auth /etc/squid/passwords
auth_param basic realm Proxy login
acl authenticated proxy_auth REQUIRED
http_access allow authenticated

sudo squid -k reconfigure
```

> *Order matters: put deny rules before the allow rule that would match the same traffic, and keep deny all at the end.*

Squid: Client Setup, Testing and Logs

TEST FROM A CLIENT

```
# Test with curl through the proxy
curl -x http://192.168.1.10:3128 -I http://example.com

# Browser or system setting
Proxy: 192.168.1.10   Port: 3128

# Service and port
systemctl status squid
ss -tlnp | grep 3128
```

READ THE ACCESS LOG

```
sudo tail -f /var/log/squid/access.log

# Typical result codes:
# TCP_MISS/200   fetched from the internet
# TCP_HIT/200    served from the cache
# TCP_DENIED/403 blocked by an acl

sudo tail /var/log/squid/cache.log
sudo squid -k rotate
```

> *A blocked site must show TCP_DENIED/403 in access.log. If it shows TCP_MISS, your deny rule is below an allow rule.*

SSH: Remote Administration

SSH

```
# Install and check
sudo apt install openssh-server
systemctl status ssh
ss -tlnp | grep :22

# Connect
ssh alice@192.168.1.10
ssh -p 2222 alice@server

# Copy files securely
scp report.pdf alice@server:/tmp/
sftp alice@server

# Validate config
sudo sshd -t
```

- **Purpose**
  Encrypted remote shell and file transfer
- **Software**
  OpenSSH (sshd), config in /etc/ssh/sshd_config
- **Port**
  TCP 22; also SFTP and port forwarding
- Most attacked service on the internet, hardened in Deck 2

DNS with BIND9

BIND9

```
# Install and validate
sudo apt install bind9
sudo named-checkconf
sudo named-checkzone example.com \
    /etc/bind/db.example.com
sudo systemctl reload bind9

# Query it
dig @localhost example.com A
dig example.com MX +short
nslookup example.com

# Follow the delegation from the root
dig +trace example.com
```

- **Purpose**
  Resolves names to IPs, UDP/TCP 53
- **Authoritative**
  Answers for your own zones
- **Recursive**
  Looks up names for clients. Restrict it to your LAN with allow-recursion.
- Never expose an open resolver to the internet

DHCP: ISC DHCP Server

DHCP

```
# Install and validate
sudo apt install isc-dhcp-server
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
sudo systemctl restart isc-dhcp-server

# /etc/dhcp/dhcpd.conf
subnet 192.168.1.0 netmask 255.255.255.0 {
  range 192.168.1.100 192.168.1.200;
  option routers 192.168.1.1;
  option domain-name-servers 192.168.1.10;
  default-lease-time 600;
  max-lease-time 7200;
}
host printer {
  hardware ethernet aa:bb:cc:dd:ee:ff;
  fixed-address 192.168.1.50;
}
```

- **Assigns**
  IP, gateway and DNS to LAN clients, UDP 67/68
- **Reservations**
  Fixed IP per MAC address for printers and servers
- **Leases**
  cat /var/lib/dhcp/dhcpd.leases
- Run only on trusted internal networks. ISC Kea is the modern successor.

File Sharing: Samba and NFS

SAMBA

```
sudo apt install samba
sudo nano /etc/samba/smb.conf
[shared]
  path = /srv/samba/shared
  valid users = @staff
  read only = no
  browseable = yes

sudo smbpasswd -a alice
testparm
sudo systemctl restart smbd
smbclient -L //server -U alice
```

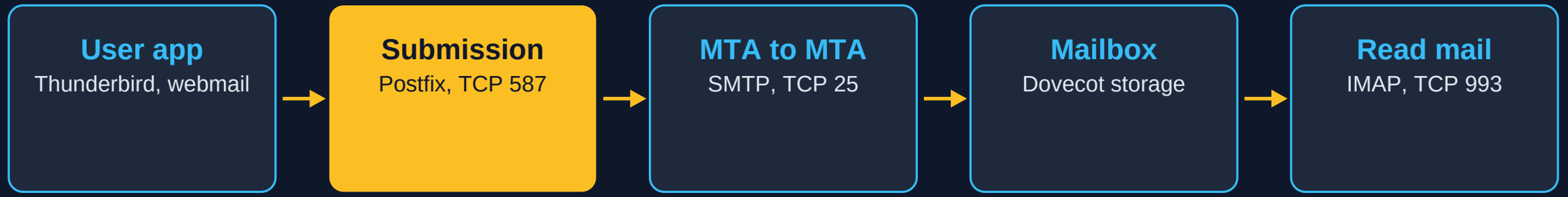
NFS

```
sudo apt install nfs-kernel-server
sudo nano /etc/exports
/srv/share 192.168.1.0/24(ro,sync,no_subtree_check)

sudo exportfs -ra
showmount -e server

# On the client
sudo mount -t nfs server:/srv/share /mnt/share
df -h /mnt/share
```

Mail: Postfix + Dovecot



- **Postfix**
  sends and routes mail. Check with postconf -n, mailq and /var/log/mail.log
- **Dovecot**
  serves mailboxes to users over IMAP/POP3 with TLS
- **DNS records**
  MX, SPF, DKIM and DMARC decide whether other servers trust you
- Misconfigured relays become spam sources, never run an open relay

Finding What Is Actually Running

AUDIT

```
# Listening ports and the program behind them
sudo ss -tulpn
sudo lsof -i -P -n | grep LISTEN

# What does the outside world see?
nmap -sV 192.168.1.10          # from another machine
nmap -p- 192.168.1.10          # all 65535 ports

# Remove what is not needed
sudo systemctl disable --now cups avahi-daemon
sudo apt purge telnetd
```

- Compare ss output (inside) with nmap (outside)
- Unknown service? Find its package with dpkg -S $(which name)
- Re-run the audit after every install or upgrade

Service Security Checklist

- **Inventory**
  know every listening service and who owns it
- **Minimize**
  disable and remove unused services
- **Bind narrowly**
  localhost or internal interfaces when possible
- **Encrypt**
  SSH, HTTPS/TLS and SFTP instead of telnet or FTP
- **Least privilege**
  run as non-root with a dedicated service user
- **Patch and monitor**
  update packages, read advisories, watch the logs (Decks 2 and 3)

Common Problems and Quick Fixes

Symptom	Likely cause	First check
Connection refused	Service down or wrong port	`systemctl status X, ss -tlnp`
Connection timed out	Firewall or routing drops it	`ufw status, traceroute`
Address already in use	Another process owns the port	`ss -tlnp | grep :PORT`
Failed to start	Bad config or permissions	`journalctl -xeu X, X -t`
Works locally, not remotely	Bound to 127.0.0.1	`listen / bind-address setting`
Permission denied (publickey)	Wrong key or file permissions	`ssh -v, chmod 700 ~/.ssh`

Mini Lab: Build the Four Required Services

- **1. Web**
  install nginx or Apache, create a virtual host, confirm with curl -I http://localhost
- **2. FTP**
  install vsftpd, create ftpuser, upload and download a file with lftp
- **3. Database**
  install MariaDB, create appdb and an app user, run a SELECT, take a mysqldump backup
- **4. Proxy**
  install Squid, point a client to port 3128, block one site, find TCP_DENIED in access.log
- **5. Audit**
  run ss -tulpn and confirm each service listens only where it should

> *Deliverable: screenshots of each test plus a table of service, port and config file.*

Summary

- **Web**
  Nginx or Apache on ports 80/443: virtual hosts, config test, TLS with certbot
- **FTP**
  vsftpd on 21 plus a passive port range: chroot users, prefer FTPS or SFTP
- **Database**
  MariaDB 3306 / PostgreSQL 5432: least-privilege users, bind to localhost, regular backups
- **Proxy**
  Squid on 3128: ACLs, authentication, caching, read access.log
- Manage with systemctl, troubleshoot with journalctl, check exposure with ss and nmap
- Fewer services means a smaller attack surface

Questions?

Next: Deck 2, Linux Server Security Hardening